ResultParser using BeautifulSoup

1. Purpose of this Code

- This code extracts student information and subject results from the HTML response of the university result page using BeautifulSoup.
- It parses two tables from the HTML:
  1) Student information table (id = "student_info")
  2) Subjects results table (id = "results_subject_table")
- Then it returns the data in a structured Python format (dictionary and list of dictionaries).

2. Import and Class Initialization

```
from bs4 import BeautifulSoup
class ResultParser:
    def __init__(self, html):
        self.html = html
        self.soup = BeautifulSoup(html, 'html.parser')
```

html → string that contains the result page html

Creates a BeautifulSoup object to parse the HTML. 'html.parser' is built-in parser.

3. parse_student_info() (Extract basic details)

```
def parse_student_info(self):
    student_tables = self.soup.find_all(
        "table", id="student_info"
    )
    if not student_tables:
        return None  # table not found
    rows = student_tables[0].find_all("tr")
    exam_name = rows[0].find_all("td")[0].get_text(" ", strip=True)
    register_text = rows[1].get_text(" ", strip=True)
    student_name = rows[2].get_text(" ", strip=True)
    register_number = register_text.split(":", 1)[1].strip()
    student_name = regiect_name.split(":", 1)[1].strip)
    return {
        "register_number": register_number,
        "student_name": student_name,
    }   "exam_name": exam_name
```

Explanation step by step:

1. find_all("table", id="student_info") → finds all <table> tags with id="student_info".
2. If table not found, return None.
3. rows = find_all("tr") → get all table rows.
4. First row → exam name (first <td>).
5. Second row → contains register number text like "Register No : 23TN0047".
6. Third row → contains student name text like "Name : Magesh T".
7. split(":", 1) → splits the text at first : and takes the part after :.

Example HTML (student_info table)

```
<table id="student_info">
 <tr>
    <td> B.Tech Degree Examination - April 2025 </td>
 </tr>
 <tr>
    <td> Register No : 23TN0047 </td>
 </tr>
 <tr>
    <td> Name : Magesh T </td>
 </tr>
</table>
```

How it extracts?

```
rows[0].find_all('td')[0]
  → "B.Tech Degree
     Examination - April 2025"
rows[1].get_text()
  → "Register No : 23TN0047"
  → split(':', 1)[1]
  → "23TN0047"
rows[2].get_text()
  → "Name : Magesh T"
  → split(':', 1)[1]
  → "Magesh T"
```

4. parse_subjects() (Extract subject results)

```
def parse_subjects(self):
    subject_table = self.soup.find('table', id="results_subject_table")
    if subject_table is None:
        return None  # table not found
    subjects = []
    rows = subject_table.find_all('tr')
    for row in rows[1:]:  # skip header row
        columns = row.find_all('td')
        subject = {
            "subject_name": columns[1].get_text(' ', strip=True),
            "type":         columns[2].get_text(' ', strip=True),
            "credit":       columns[3].get_text(' ', strip=True),
            "grade_point":  columns[4].get_text(' ', strip=True),
            "credit_point": columns[5].get_text(' ', strip=True),
            "grade":        columns[6].get_text(' ', strip=True),
        }
        subjects.append(subject)
    return subjects
```

find() → returns first matching tag. Here we expect only one table.

rows[1:] → start from second row because first row is header (column names).

find_all('td') → gets all columns in that row.

For each row, we extract required columns and store in a dictionary. Then append it to the subjects list.

Example HTML (results_subject_table)

```
<table id="results_subject_table">
 <tr>
    <th> S.No</th> <th>Subject </th> <th>Type </th> <th>Credit</th>
    <th> Grade Point </th> <th>Credit Point</th> <th>Grade</th>
 </tr>
 <tr>
    <td> 1</td> <td> AI &ML </td> <td>Theory </td> <td>4 </td>
    <td> 8 </td> <td> 32 </td> <td> A </td>
 </tr>
</table>
```

Extracted data (dictionary)

```
{
 'subject_name': 'AI & ML',
 'type': 'Theory',
 'credit': '4',
 'grade_point': '8',
 'credit_point': '32',
 'grade': 'A'
}
```

5. Main Program (Example Usage)

```
if __name__ == "__main__":
    fetcher = ResultFetcher("23TN0047", SEMESTER_EXAM_CODES[6])
    response = fetcher.fetch_result()
    if response and response["error_code"] != -1:
        html = response["data"]["html"]
        parser = ResultParser(html)
        student = parser.parse_student_info()
        subjects = parser.parse_subjects()
        print(student)
        for subject in subjects:
            print(subject["subject_name"], "->", subject["grade"], ...)
    else:
        print("Result not available.")
```

Flow:

1. Fetch HTML using ResultFetcher.
2. Check if response is success.
3. Create ResultParser with HTML.
4. Extract student info and subjects.
5. Print the details.

ResultFetcher using requests (Python)

Fetch University Result using HTTP Requests

1. Purpose

- This class sends HTTP requests to the university website to get the student result.
- It first opens the normal result page (to create a session and get cookies) and then calls the actual result API to get the result in JSON format.
- It uses `requests.Session()` to maintain cookies between requests.

2. Required Imports and Configuration

```
import requests
from config import BASE_URL, RESULT_API_URL,
                   SEMESTER_EXAM_CODES
```

These are defined in config.py
- BASE_URL → main result page (result.php)
- RESULT_API_URL → actual API (app.php?a=DisplayStudentResult)
- SEMESTER_EXAM_CODES → mapping (1:A, 2:B, ⋯ 6:F)

3. Class Initialization (__init__)

```
class ResultFetcher:
    def __init__(self, register_number, exam_code):
        self.register_number = register_number
        self.exam_code = exam_code
        # create a session
        self.session = requests.Session()
```

Store register number and exam code (e.g. "23TN0047", "F")

requests.Session() → maintains cookies automatically between multiple requests.

4. fetch_result() - Step by Step Explanation

```
def fetch_result(self):
    params = { "r": self.register_number,
               "e": self.exam_code }
    data = { "r": self.register_number,
             "e": self.exam_code,
             "ct": "" }
```

- params → used in GET request. Added to URL (query string)
- data → used in POST request. Sent in request body. "ct" is an extra parameter required by the server.

STEP 1: Open normal result page (GET request)

```
page_response = self.session.get(
    BASE_URL,
    params = params,
    timeout = 15
)
page_response.raise_for_status()
```

- Sends GET request to result.php with r and e as query parameters.
- Server creates a session and returns cookies.
- raise_for_status() → raises an exception if status code is not 200 (e.g. 404, 500).

Overall Flow

Step 1 GET result.php (with params) → Server creates session and sends cookies → Step 2 POST app.php?a=DisplayStudentResult (with data + headers) → Server returns JSON result → Convert to dictionary and return

STEP 2: Call actual result API (POST request)

```
headers = {
    "X-Requested-With": "XMLHttpRequest",
    "Referer": page_response.url
}
response = self.session.post(
    RESULT_API_URL,
    data = data,
    headers = headers,
    timeout = 15
)
response.raise_for_status()
```

- X-Requested-With: XMLHttpRequest → tells the server that this is an Ajax/JavaScript request. The server returns JSON instead of full HTML.
- Referer → set to the URL of the first request.

- Sends POST request to app.php with form data (r, e, ct) and headers.
- Uses the same session, so cookies from Step 1 are automatically sent.
- raise_for_status() → checks for HTTP errors.

STEP 3: Convert JSON response to Python dictionary

```
result = response.json()
return result
```

- The server returns JSON data like: { "error_code": 0, "status": "success", "data": {"html": "...(result page html)..."}}
- response.json() converts JSON text to a Python dictionary.

Exception Handling

```
except requests.RequestException as error:
    print(f"Error fetching result: {error}")
    return None
```

- Catches network errors, timeouts, HTTP errors, etc. and returns None.

5. Main Program (Example Usage)

```
if __name__ == "__main__":
    result = ResultFetcher("23TN0047", SEMESTER_EXAM_CODES[6])
    fetched_result = result.fetch_result()

    if fetched_result:
        print("Result fetched successfully!")
        print(fetched_result)
    else:
        print("Failed to fetch the result.")
```

Create a ResultFetcher object with register number and semester code (e.g. 6 → 'F')

- Call fetch_result() and print the result (you can process it further using a parser).

6. URL Construction using f-strings (Alternative)

```
register_number = "23TN0047"
semester_number = 6
semester_code_mapping = {
    1: "A", 2: "B", 3: "C", 4: "D",
    5: "E", 6: "F", 7: "G", 8: "H"
}
semester_code = semester_code_mapping[semester_number]
url = f"https://exam.pondiuni.edu.in/results/result.php?r={register_number}&e={semester_code}"
```

This creates the same URL as using params in the get() request.
Example:
https://exam.pondiuni.edu.in/results/result.php?r=23TN0047&e=F

7. Key Concepts Used

Concept	Description
requests.get()	Sends a GET request (params added to URL).
requests.post()	Sends a POST request (data sent in request body).
requests.Session()	Maintains cookies between multiple requests.
params	Dictionary of query parameters for GET request.
data	Dictionary of form data for POST request.
headers	Additional HTTP headers (e.g. X-Requested-With, Referer).
response.json()	Converts JSON responss text to Python dictionary.
raise_for_status()	Raises an exception for HTTP error codes (4xx, 5xx).
try...except	Handles network errors and prevents the program from crashing.

UniversityResultAutomator (main.py)

Automate fetching university results for a range of register numbers and save to Excel using Python.

1. Importing Modules

```
import os
import re
import time

from concurrent.futures import ThreadPoolExecutor, as_completed
from tqdm import tqdm

from config import SEMESTER_EXAM_CODES
from result_fetcher import ResultFetcher
from result_parser import ResultParser
from excel_generator import ExcelGenerator
```

Purpose of each module

- os → create folder, file operations
- re → regular expressions (to split register number)
- time → delay between requests
- ThreadPoolExecutor → for multithreading (concurrent fetching)
- tqdm → progress bar
- config → semester code mapping (1:A, 2:B, ... 6:F)
- ResultFetcher → fetch result from university website
- ResultParser → extract student info and subject details from HTML
- ExcelGenerator → save results into Excel file

2. Class Initialization (__init__)

```
class UniversityResultAutomator:
    def __init__(self):
        self.start_register = None
        self.end_register = None
        self.semester = None
        self.exam_code = None

        self.register_numbers = []   # list of all registers
        self.results = []            # list to store fetched results
```

- Stores user inputs and required data as class variables.
- register_numbers → all register numbers to process.
- results → final parsed results (before saving to Excel).

3. get_user_input() - Take input from user

```
def get_user_input(self):
    self.start_register = input("Enter the starting Register Number: ")
    self.end_register = input("Enter the ending Register Number: ")
    try:
        self.semester = int(input("Enter your Semester (1-8): "))
    except ValueError:
        print("Semester must be an integer. Please enter a number between 1 and 8.")
        return False
    if self.semester not in SEMESTER_EXAM_CODES:
        print("Invalid semester. Please enter a number between 1 and 8.")
        return False
    self.exam_code = SEMESTER_EXAM_CODES[self.semester]
    return True
```

- Takes starting and ending register number and semester from user.
- Validates semester (1 to 8).
- Gets exam_code from SEMESTER_EXAM_CODES (e.g. 6 → "F").
- Returns True if input is valid, otherwise False.

4. generate_register_numbers() - Create list of register numbers

```
def generate_register_numbers(self):
    start_match = re.match(r"^(.*?)(\d+)$", self.start_register)
    end_match = re.match(r"^(.*?)(\d+)$", self.end_register)
    if not start_match or not end_match:
        print("Invalid register number format. Please ensure they end with numbers.")
        return False
    start_prefix = start_match.group(1)
    end_prefix = end_match.group(1)
    start_number_text = start_match.group(2)
    end_number_text = end_match.group(2)
    if start_prefix != end_prefix:
        print("Register number prefixes do not match.")
        return False
    start_number = int(start_number_text)
    end_number = int(end_number_text)
    if start_number > end_number:
        print("Starting register number must be less than or equal to ending.")
        return False
    number_length = len(start_number_text)
    for number in range(start_number, end_number + 1):
        register_number = (start_prefix + str(number).zfill(number_length)).upper()
        self.register_numbers.append(register_number)
    print("\nTotal Register Numbers: ", len(self.register_numbers))
    return True
```

- Uses regular expression: `^(.*?)(\d+)$`
  - `(.*?)` → prefix (non-greedy)
  - `(\d+)` → last numbers

  Example: "23TN0047" → prefix: "23TN", number: "0047"
- Checks both register numbers have same prefix (e.g. 23TN).
- Extracts starting and ending number part and converts to int.
- Ensures start ≤ end.
- Uses zfill(number_length) to maintain leading zeros. Example: 47 → 0047
- Generates all register numbers in the range and stores in self.register_numbers.

Example:
Start : 23TN0047
End : 23TN0050
Generated List:
[23TN0047, 23TN0048, 23TN0049, 23TN0050]

5. process_students() - Fetch and parse results (Sequential Version)

```
def process_students(self):
    total = len(self.register_numbers)
    for index, register_number in enumerate(self.register_numbers, start=1):
        print(f"\n[{index}/{total}] Processing Register Number : {register_number}")
        fetcher = ResultFetcher(register_number, self.exam_code)
        response = fetcher.fetch_result()
        if not response:
            print(f"Failed to fetch result for {register_number}. Skipping...")
            continue
        if response.get("error_code") == -1:
            print(register_number, "->", response.get("data",{}).get("html","Result not available."))
            continue
        try:
            html = response["data"]["html"]
            parser = ResultParser(html)
            student = parser.parse_student_info()
            subjects = parser.parse_subjects()
            if student is None:
                print(f"Student details not found for {register_number}.")
            if not subjects:
                print(f"Subjects details not found for {register_number}.")
                continue
            self.results.append({ "student" : student
                                  "subjects" : subjects})
            print(f"Result fetched and parsed successfully for {register_number}.")
        except Exception as error:
            print(f"An error occurred while processing {register_number}: {error}")
    time.sleep(0.5)  # delay between requests to avoid overwhelming the server
```

- Loops through each register number one by one.
- Creates ResultFetcher → call university API.
- Checks for errors or unavailable results.
- Parses HTML using ResultParser → get student info and subject details.
- If successful, appends to self.results.
- Adds 0.5 seconds delay between requests to avoid server overload.
- Returns True after processing all.

Multithreading Version (commented in code)

- Uses ThreadPoolExecutor to fetch multiple results concurrently.
- tqdm shows a progress bar.
- Helps to reduce total time but may increase load on server.

6. generate_excel() - Save results to Excel

```
def generate_excel(self):
    if not self.results:
        print("No results to generate Excel. Please fetch results first.")
        return
    os.makedirs("output", exist_ok=True)
    file_name = f"output/University_Results_{self.semester}.xlsx"
    excel_generator = ExcelGenerator(file_name)
    excel_generator.create_excel(self.results, self.semester)
```

- Checks if any results are available.
- Creates 'output' folder (if not exists).
- File name includes semester number.
- Uses ExcelGenerator class to create and save the Excel file.

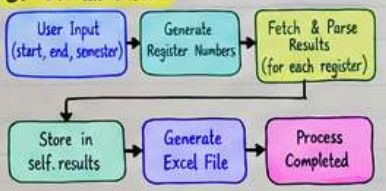
7. run() - Main Function

```
def run(self):
    if not self.get_user_input():
        return
    if not self.generate_register_numbers():
        return

    self.generate_excel()
```

```
    self.process_students()
    print("\nSuccessfully Fetched :", len(self.results))
    print("Failed/Unavailable :", len(self.register_numbers) - len(self.results))
    print("\nProcess completed successfully!")
```

8. Overall Flow



Multithreading in Python (Detailed Explanation)

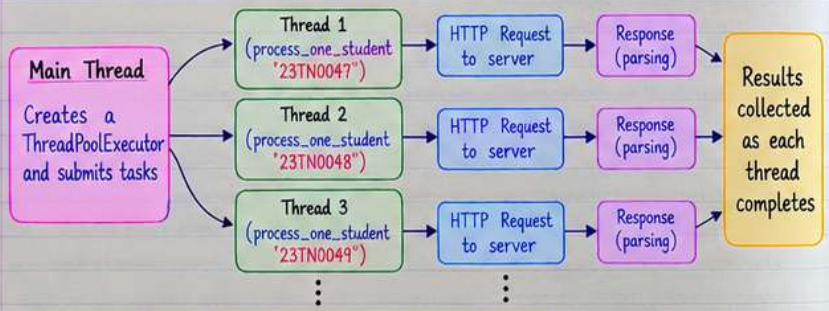
1. What is Multithreading ?

- Multithreading means running multiple tasks (threads) at the same time within a single program.
- In Python, we use ThreadPoolExecutor from concurrent.futures to easily create and manage threads.
- In this application, each thread will fetch and parse one student result.
- While one thread is waiting for the server response, other threads can process different register numbers. This reduces total time.

2. Why use Multithreading here ?

- Each request to university server takes some time (waiting for response).
- During this waiting time, CPU is mostly idle.
- Using multiple threads, we can send several requests simultaneously and wait for their responses, reducing total time.

3. How it works ? (Step by Step)



4. Actual Code (Multithreading Version)

```python
from concurrent.futures import ThreadPoolExecutor, as_completed
from tqdm import tqdm

def process_one_student(self, register_number):
    try:
        fetcher = ResultFetcher(register_number, self.exam_code)
        response = fetcher.fetch_result()
        if not response:
            print(f"Failed to fetch result for {register_number}")
            return None

        if response.get("error_code") == -1:
            print(register_number, "->",
            response.get("data",{}).get("html","Result not available."))
            return None
        html = response["data"]["html"]
        parser = ResultParser(html)
        student = parser.parse_student_info()
        subjects = parser.parse_subjects()

        if not student or not subjects:
            return None
        return {"student": student, "subjects": subjects}
    except Exception as e:
        print(f"Error processing {register_number}: {e}")
        return None
```

Function that processes a single register number. This function will run in a separate thread for each register number.

Returns a dictionary with student and subjects if successful. Otherwise returns None.

Any exception is caught and None is returned.

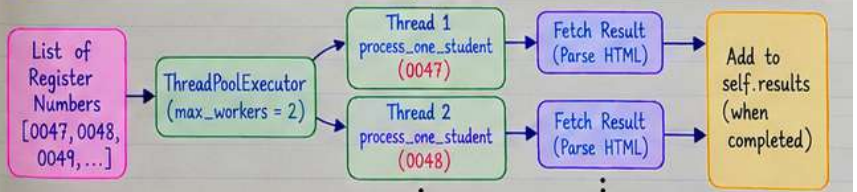
5. process_students() using Multithreading

```python
def process_students(self):
    total = len(self.register_numbers)
    print(f"\nProcessing {total} students...")
    # Create a thread pool with 2 worker threads
    with ThreadPoolExecutor(max_workers=2) as executor:
        # Submit a task for each register number
        futures = {
            executor.submit(self.process_one_student, register_number): register_number
            for register_number in self.register_numbers
        }
    # Use tqdm to show progress bar
    with tqdm(total=total, desc="Fetching Results", unit="student") as progress_bar:
        for future in as_completed(futures):  # get completed tasks as soon as they finish
            register_number = futures[future]  # which register number this future is for
            try:
                result = future.result() # get the return value from the thread
                if result:
                    self.results.append(result)
            except Exception as e:
                print(f"Error processing {register_number}: {e}")
            progress_bar.update(1)  # increase progress bar by 1
    return True
```

6. Explanation of Important Parts

Code / Concept	Explanation
ThreadPoolExecutor(max_workers=2)	Creates a pool of 2 threads. Only 2 requests will be processed at a time.
executor.submit(func, arg)	Submits a function to be executed in a separate thread. Returns a Future object.
futures = { ... }	Dictionary mapping each Future to its register number.
as_completed(futures)	Yields futures as they complete (not in the original order).
future.result()	Returns the result from the thread (whatever process_one_student returned). If the thread raised an exception, it will raise it here.
tqdm(...)	Displays a progress bar in the terminal.
progress_bar.update(1)	Increases the progress bar by 1 for each completed task.

7. Visual Flow of Multithreading Code



8. Things to Remember

- Use a small number of threads (e.g., 2-5) to avoid server overload.
- Each thread runs process_one_student() independently.
- as_completed() gives results as soon as they are finished.
- Progress bar works even when using multithreading.
- Results may be returned in a different order than input, which is fine.
- Handle exceptions properly so one failed thread does not stop others.

9. Sequential vs Multithreading

Aspect	Sequential (for loop)	Multithreading
Execution	One by one	Multiple at same time
Total time	Higher (sum of all times)	Lower (faster)
Code	Simple	Slightly complex
Server load	Low	Medium (use small max_workers)
Order of results	Same as input	May be different
Use case	Small number of registers	Large number of registers

Multithreading Code – Real Time Execution Flow

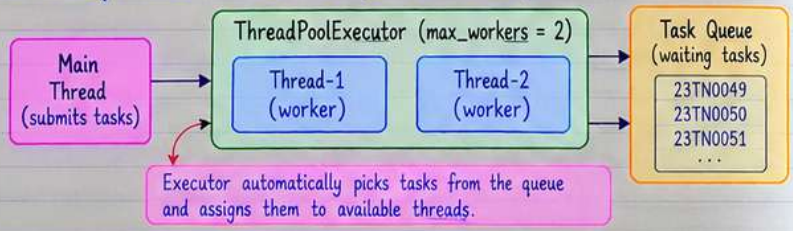
1. Overview of the code (main part)

```
with ThreadPoolExecutor(max_workers = 2) as executor:
    futures = {
        executor.submit(self.process_one_student, reg_no): reg_no
        for reg_no in self.register_numbers
    }
    for future in as_completed(futures):
        reg_no = futures[future]
        result = future.result()
        if result:
            self.results.append(result)
        progress_bar.update(1)
```

- create a pool of 2 threads manage automatically
- submit a task for each register number and store future object
- iterate futures in the order they complete (not submission order)
- get the register number for this future
- get the result returned by the thread
- store the result
- update progress bar

2. What is ThreadPoolExecutor ?

- It creates a pool of worker threads.
- max_workers = 2 → only 2 threads will run at the same time.
- It manages thread creation, task assignment, and cleanup.



3. What happens in executor.submit() ?

```
future = executor.submit(self.process_one_student, "23TN0047")
```

- submits the function and its arguments to a worker thread.
- returns a Future object immediately (task ticket).
- The function starts running in a separate thread.
- Does not wait for the result.

Future object (represents running task)

- future.done() → False (initially)
- future.result() → wait and return result
- future.exception() → error (if any)

4. Creating futures dictionary

```
futures = {
    executor.submit(self.process_one_student, reg_no): reg_no
    for reg_no in self.register_numbers
}
```

- For each register number, a task is submitted and a Future object is returned.
- We store the Future as key and register number as value.
- This helps us know which register number belongs to which future when it completes.

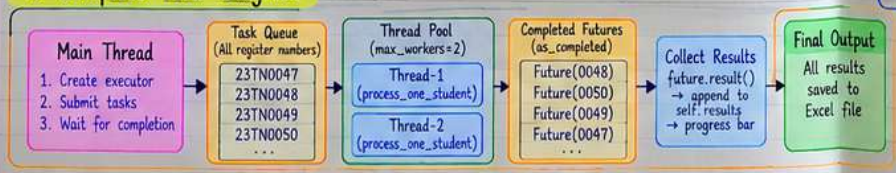
Example:

```
self.register_numbers = ["23TN0047", "23TN0048", "23TN0049"]
```

futures dictionary

```
<Future at 0x1a2b> : "23TN0047"
<Future at 0x3c4d> : "23TN0048"
<Future at 0x5e6f> : "23TN0049"
```

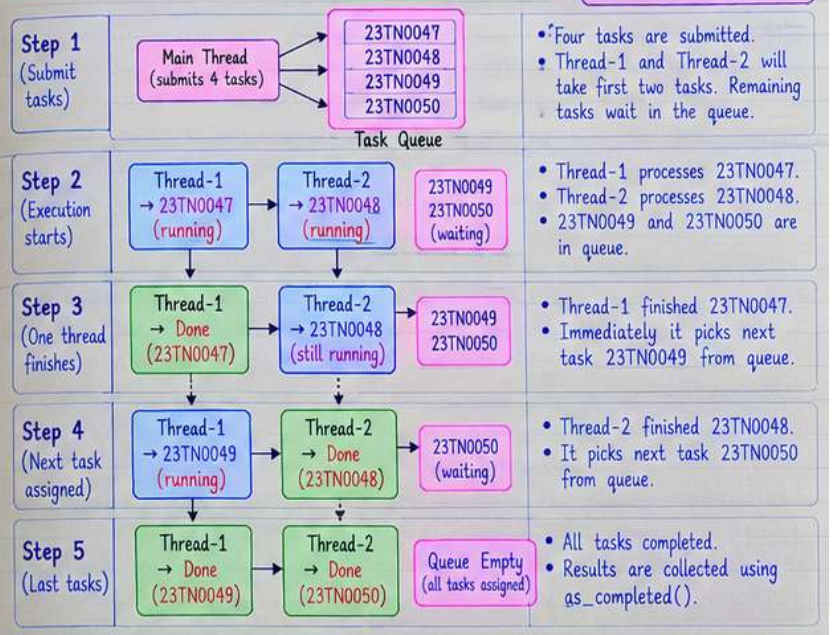
9. Complete Flow Diagram



5. Real Time Execution Example (Step by Step)

Register Numbers : [23TN0047, 23TN0048, 23TN0049, 23TN0050]

max_workers = 2 (only 2 threads at a time)



6. What is as_completed() ?

- It yields futures in the order they complete' (not the order they were submitted).
- Helps us process results immediately when a task finishes.
- Useful when tasks take different time to complete.

Example (Completion order)

23TN0047 → 5 sec
23TN0048 → 2 sec
23TN0049 → 4 sec
23TN0050 → 3 sec

as_completed() gives futures in this order:
23TN0048 → 23TN0050 → 23TN0049 → 23TN0047
(based on actual completion time)

7. Inside the loop with as_completed()

```
for future in as_completed(futures):
    reg_no = futures[future]   # get register number
    result = future.result()   # wait and get result
    if result:
        self.results.append(result)   # store result
    progress_bar.update(1)        # update tqdm
```

1. Get the next completed future.
2. Find the corresponding register number using futures[future].
3. future.result() returns the value returned by process_one_student().
4. Append to self.results if valid.
5. Update progress bar.

8. Meaning of "with"

```
with ThreadPoolExecutor(max_workers = 2) as executor:
```

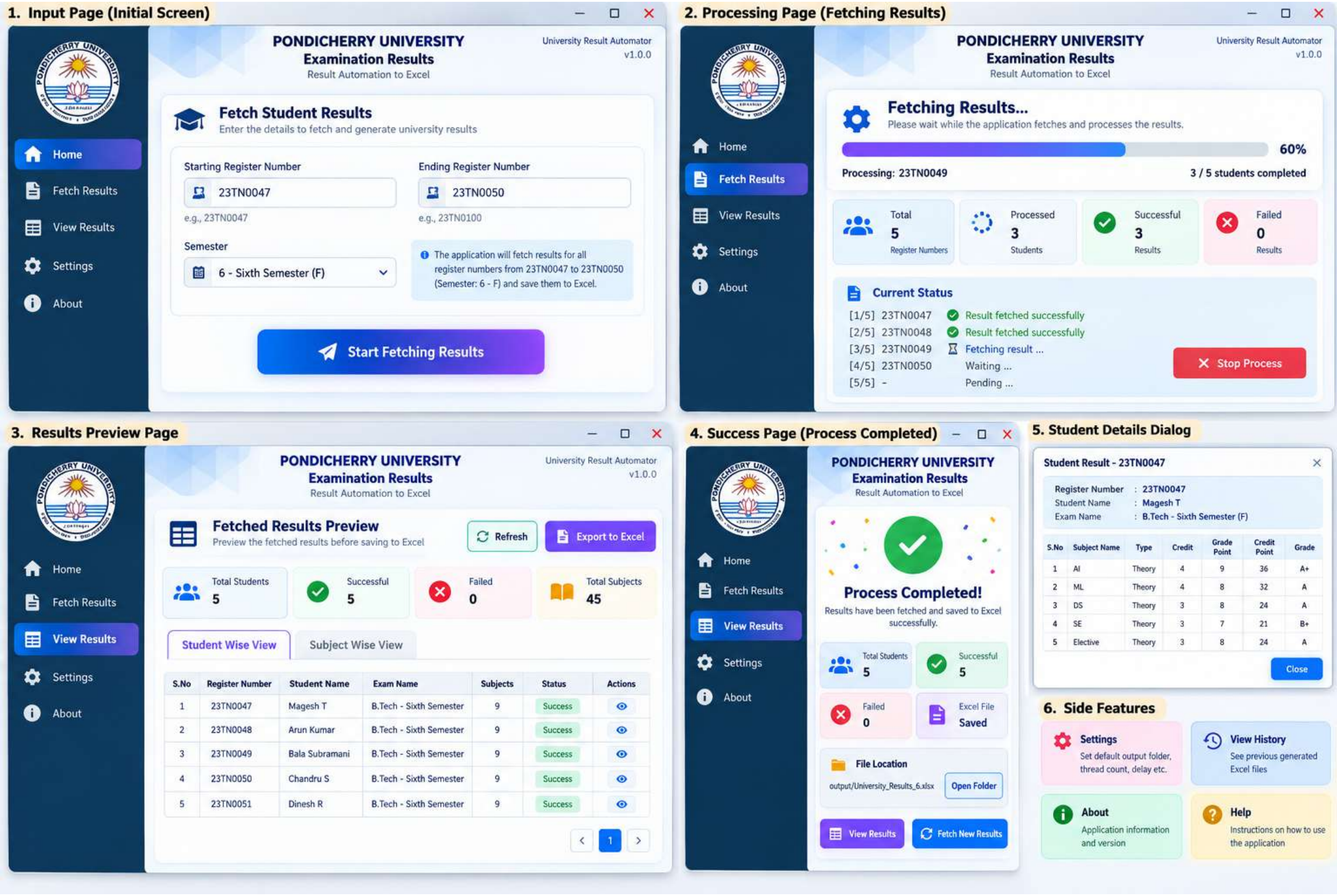
- "with" is a context manager.
- It automatically calls executor.shutdown() at the end.
- This ensures all threads are properly closed, even if an error occurs.
- We don't need to manually call executor.shutdown().

Equivalent code without "with"

```
executor = ThreadPoolExecutor(max_workers=2)
try:
    # submit tasks and process
finally:
    executor.shutdown()  # close threads
```

9. Key Points

- executor → manages a pool of threads.
- submit() → sends a function to run in a thread and returns a Future.
- Future → represents a running task and holds its result.
- as_completed() → gives futures as they finish (faster results processing).
- with → automatically cleans up the executor (closes threads).
- max_workers → number of concurrent threads.
- This reduces total time and avoids overloading the server.


1. Input Page (Initial Screen)
PONDICHERRY UNIVERSITY
Examination Results
Result Automation to Excel
University Result Automator
v1.0.0
Home
Fetch Results
View Results
Settings
About
Fetch Student Results
Enter the details to fetch and generate university results
Starting Register Number
23TN0047
e.g., 23TN0047
Ending Register Number
23TN0050
e.g., 23TN0100
Semester
6 - Sixth Semester (F)
The application will fetch results for all register numbers from 23TN0047 to 23TN0050 (Semester: 6 - F) and save them to Excel.
Start Fetching Results
2. Processing Page (Fetching Results)
PONDICHERRY UNIVERSITY
Examination Results
Result Automation to Excel
University Result Automator
v1.0.0
Home
Fetch Results
View Results
Settings
About
Fetching Results...
Please wait while the application fetches and processes the results.
60%
Processing: 23TN0049
3 / 5 students completed
Total
5
Register Numbers
Processed
3
Students
Successful
3
Results
Failed
0
Results
Current Status
[1/5] 23TN0047 Result fetched successfully
[2/5] 23TN0048 Result fetched successfully
[3/5] 23TN0049 Fetching result ...
[4/5] 23TN0050 Waiting ...
[5/5] - Pending ...
Stop Process
3. Results Preview Page
PONDICHERRY UNIVERSITY
Examination Results
Result Automation to Excel
University Result Automator
v1.0.0
Home
Fetch Results
View Results
Settings
About
Fetched Results Preview
Preview the fetched results before saving to Excel
Refresh
Export to Excel
Total Students
5
Successful
5
Failed
0
Total Subjects
45
Student Wise View
Subject Wise View
S.No
Register Number
Student Name
Exam Name
Subjects
Status
Actions
1 23TN0047 Magesh T B.Tech - Sixth Semester 9 Success
2 23TN0048 Arun Kumar B.Tech - Sixth Semester 9 Success
3 23TN0049 Bala Subramani B.Tech - Sixth Semester 9 Success
4 23TN0050 Chandru S B.Tech - Sixth Semester 9 Success
5 23TN0051 Dinesh R B.Tech - Sixth Semester 9 Success
1
4. Success Page (Process Completed)
PONDICHERRY UNIVERSITY
Examination Results
Result Automation to Excel
Home
Fetch Results
View Results
Settings
About
Process Completed!
Results have been fetched and saved to Excel successfully.
Total Students
5
Successful
5
Failed
0
Excel File
Saved
File Location
output/University_Results_6.xlsx
Open Folder
View Results
Fetch New Results
5. Student Details Dialog
Student Result - 23TN0047
Register Number : 23TN0047
Student Name : Magesh T
Exam Name : B.Tech - Sixth Semester (F)
S.No
Subject Name
Type
Credit
Grade Point
Credit Point
Grade
1 AI Theory 4 9 36 A+
2 ML Theory 4 8 32 A
3 DS Theory 3 8 24 A
4 SE Theory 3 7 21 B+
5 Elective Theory 3 8 24 A
Close
6. Side Features
Settings
Set default output folder, thread count, delay etc.
View History
See previous generated Excel files
About
Application information and version
Help
Instructions on how to use the application